

React Lesson 3: Props & State — Complete Study Guide

1: Course Roadmap & Core Concepts

- **Course:** Modern React Architecture Series
- **Topic:** Lesson 3 — Props & State

Curriculum Overview

- **Props Fundamentals:** Unidirectional data flow, passing data, read-only guarantees.
- **Destructuring & Patterns:** Cleaner parameter syntax, default values, rest props, and children.

- **State & useState Hook:** Component memory, triggering re-renders, snapshot concept.
- **State Updates:** Asynchronous queueing, handling stale state with updater callbacks (prev => prev + 1).
- **Complex Data Handling:** Managing multiple states, immutability rules for arrays and objects.
- **Form & Events:** SyntheticEvent system, controlled vs uncontrolled inputs.

Core Rule: Props make components pure with respect to inputs; State allows components to change UI over time in response to user action.

2: Deep Dive into Props

What are Props? Props (properties) are read-only inputs passed from a parent component down to a child component. They represent React's unidirectional (top-down) data flow.

Key Rules:

- **Immutable:** A child cannot alter the props it receives.
- **Deterministic:** Identical props and state will always yield the exact same UI.
- **Flexible:** Props can accept strings, numbers, objects, arrays, functions, and JSX elements.

JavaScript

// Passing multiple prop types

```
function Parent() {
```

```
  const user = { name: "Sarah", role:  
  "Admin" };
```

```
  const skills = ["React", "Node.js"];
```

```
  return (
```

```
    <UserCard
```

```
      user={user}
```

```
      skills={skills}
```

```
      onLogout={() =>
```

```
        console.log("Logged out")}
```

```
    />
```

```
  );
```

```
}
```

3: Props Destructuring & Composition

Instead of using props.property repeatedly, modern React uses ES6 destructuring directly in the component signature.

Destructuring & Default Values:

JavaScript

```
// With default value fallback
```

```
function UserBadge({ name = "Guest",  
role = "Member", avatarUrl }) {
```

```
  return (
```

```
    <div className="badge">
```

```
      <img src={avatarUrl ||
```

```
"/default.png"} alt={name} />
```

```
    <h3>{name} ({role})</h3>
  </div>
);
}
```

The children Prop: Allows components to wrap around nested content, enabling composition.

JavaScript

```
function CardContainer({ title, children
}) {
  return (
    <section className="card">
      <h2>{title}</h2>
      <div
        className="content">{children}</div>
```

```
</section>  
  
);  
  
}
```

4: State & The useState Hook

Why regular variables do not work for UI updates:

- **Reset on render:** Local variables (let count = 0) re-initialize on every function call.
- **No reactivity signal:** React does not detect variable mutation to trigger a re-render.

Props vs State vs Local Variables:

Feature	Props	State	Local Variables
Origin	Passed from parent	Created within component	Declared in function scope
Mutability	Read-Only	Mutable via setter only	Freely mutable
Triggers Re-render?	Yes (via parent)	Yes	No

Feature	Props	State	Local Variables
	t)		
Survives Re-render?	Yes	Yes (managed by React)	No (resets every time)

Anatomy of useState:

JavaScript

```
import { useState } from 'react';
```

```
function Counter() {
```

```
const [count, setCount] = useState(0);  
// [currentValue, setterFunction]
```

```
return (  
  <button onClick={() =>  
    setCount(count + 1)}>  
    Clicked: {count} times  
  </button>  
);  
}
```

5: Asynchronous Updates & Previous State Pattern

The Stale Closure Problem: React state operates on snapshots. Calling `setCount(count + 1)` multiple times in the same cycle uses the same snapshot value, resulting in only a single increment.

The Functional Updater Solution: Pass a callback `(prev) => prev + 1` to read from the internal state update queue:

JavaScript

```
function Incrementer() {  
  const [count, setCount] = useState(0);
```

```
const addThree = () => {  
  setCount(prev => prev + 1);  
  setCount(prev => prev + 1);  
  setCount(prev => prev + 1);  
  // Correctly adds 3  
};  
  
return <button  
  onClick={addThree}>Add 3:  
  {count}</button>;  
}
```

- **React 18 Batching:** Updates triggered inside timeouts, promises, or async tasks are batched into a

single render pass to maintain optimal performance.

6: Immutability with Objects & Arrays

Never mutate state directly
(`state.push()` or `state.name = "X"` won't notify React properly).

Updating Objects (Spread Operator):

JavaScript

```
const [profile, setProfile] = useState({  
  name: "Aman", score: 10 });
```

// Correct way:

```
setProfile(prev => ({  
  ...prev,  
  score: prev.score + 5
```

```
}});
```

Updating Arrays:

- **Add item:** `setList(prev => [...prev, newItem])`
- **Remove item:** `setList(prev => prev.filter((_, index) => index !== targetIndex))`
- **Update item:** `setList(prev => prev.map((item, index) => index === targetIndex ? updatedItem : item))`

7: Events & Controlled Components

Event Handling:

- React events use camelCase (onClick, onChange, onSubmit).
- Pass a function reference: onClick={handleClick}, never invoke directly onClick={handleClick()}.

Controlled Input Pattern: The input value is driven directly by state, making React the single source of truth.

JavaScript

```
function LoginForm() {  
  const [email, setEmail] = useState("");  
  
  const handleSubmit = (e) => {
```

```
e.preventDefault(); // Prevents full  
page reload
```

```
console.log("Submitted:", email);  
};
```

```
return (
```

```
<form onSubmit={handleSubmit}>
```

```
<input
```

```
type="email"
```

```
value={email}
```

```
onChange={(e) =>
```

```
setEmail(e.target.value)}
```

```
placeholder="Enter email"
```

```
/>
```



```
    <button  
type="submit">Submit</button>  
  
    </form>  
  
    );  
}
```

8: Integrated Real-World Implementation

JavaScript

```
import React, { useState } from 'react';
```

```
// Child component with destructured  
props
```

```
function TaskItem({ id, text, done,  
onToggle }) {  
    return (
```

```
<li style={{ textDecoration: done ?  
"line-through" : "none" }}>  
  <span>{text}</span>  
  <button onClick={() =>  
onToggle(id)}>  
    {done ? "Undo" : "Complete"}  
  </button>  
</li>  
);  
}  
  
// Parent component managing state  
export default function TodoApp() {  
  const [input, setInput] = useState("");
```

```
const [tasks, setTasks] = useState([
  { id: 1, text: "Learn Props", done: true
},
  { id: 2, text: "Master State", done:
false }
]);

const addTask = (e) => {
  e.preventDefault();
  if (!input.trim()) return;
  setTasks(prev => [...prev, { id:
Date.now(), text: input, done: false }]);
  setInput("");
};
```

```
const toggleTask = (id) => {  
  setTasks(prev => prev.map(task =>  
    task.id === id ? { ...task, done:  
!task.done } : task  
  ));  
};  
  
return (  
  <div>  
    <form onSubmit={addTask}>  
      <input  
        value={input}
```

```
        onChange={(e) =>
setInput(e.target.value)}

        placeholder="New task..."

    />

    <button
type="submit">Add</button>

</form>

<ul>

    {tasks.map(task => (
        <TaskItem key={task.id} {...task}
onToggle={toggleTask} />
    )))

</ul>
```

```
</div>  
  
);  
  
}
```

9: Practice Questions (1 - 10)

Q1. What is the fundamental property of props in React?

- A) Read-only and immutable by the child
- B) Mutable by the child
- C) Global variables
- D) Cleared on every render

Q2. What direction does data flow in React?

- A) Bottom-to-top

- B) Unidirectional (Parent to child)
- C) Bidirectional (Two-way)
- D) Random

Q3. What does useState return?

- A) An object with properties
- B) An array of [stateValue, setterFunction]
- C) A single update function
- D) A DOM node reference

Q4. Why is props destructuring useful?

- A) Improves performance
- B) Makes code cleaner and explicitly declares expected inputs
- C) Allows mutation of props

D) Prevents component re-rendering

Q5. What happens when a state setter function is called?

A) The state variable is immediately altered synchronously

B) React schedules a re-render of the component

C) The browser window refreshes

D) Props are cleared

Q6. When should you use the callback form `setCount(prev => prev + 1)`?

A) Only for network requests

B) When the new state depends on the immediate previous state

C) Only inside class components

D) When updating strings

Q7. What is the correct syntax for an input event listener in React?

A) onChange={handler}

B) onChange={handler}

C) onChange={handler()}

D) on-change={handler}

Q8. What is a controlled component?

A) A component that uses Redux

B) An input whose value is governed by React state

C) A component with error boundaries

D) An input that relies directly on the browser DOM

Q9. Which prop represents content enclosed inside component opening and closing tags?

- A) props.content
- B) props.children
- C) props.nested
- D) props.elements

Q10. What is the safe way to append an item to an array in state?

- A) arr.push(item); setArr(arr);
- B) setArr([...arr, item])
- C) arr[arr.length] = item; setArr(arr);
- D) setArr(arr.reverse())

Page 10: Practice Questions (11 - 20) & Answer Key

Q11. Why do normal local variables (let x = 0) fail to maintain state across re-renders?

- A) React crashes when finding local variables
- B) They reset on every render and mutations don't notify React to re-render
- C) Local variables cannot hold numbers
- D) Browsers clear them automatically

Q12. What does React 18 automatic batching do?

- A) Automatically cancels API requests
- B) Groups multiple state updates across async tasks into a single re-render

- C) Deletes unused state variables
- D) Executes setters synchronously

Q13. How do you update a single field in a state object { name: '', age: 0 }?

- A) `state.age = 20; setState(state);`
- B) `setState({ age: 20 })`
- C) `setState(prev => ({ ...prev, age: 20 }))`
- D) `setState(age = 20)`

Q14. Why is `onClick={handleClick}` used instead of `onClick={handleClick()}`?

- A) The latter executes immediately on render rather than waiting for user interaction
- B) React throws a syntax error

C) Parentheses are forbidden in JSX

D) The former is slower

Q15. What is the purpose of `e.preventDefault()` in form submission?

A) Halts network requests

B) Prevents the browser from reloading the page on submit

C) Resets input elements

D) Deletes component state

Q16. What is a React SyntheticEvent?

A) An automated UI test mock

B) A cross-browser wrapper around the native browser event

C) A performance monitoring tool

D) An artificial timer

Q17. How do you provide a default value to a destructured prop?

A) `function Card({ title = "Untitled" })`

B) `function Card(props.title || "Untitled")`

C) `function Card({ title := "Untitled" })`

D) `function Card(default title)`

Q18. How should state be shared between two sibling components?

A) Read the state directly from the sibling component

B) Lift the state up to their closest common parent

C) Store it on the global window object

D) Store it in local text files

Q19. What is a stale closure in React?

A) An open network socket

B) A function or callback capturing an outdated state value from a previous render

C) An unclosed HTML tag

D) A memory leak from CSS

Q20. What is the primary difference between Props and State?

A) Props are private; State is public

B) Props configure components externally; State is internal component memory

C) Props trigger re-renders; State does not

D) State cannot hold objects

Answer Key

Q#	Answer	Explanation / Core Concept
Q1	A	Props are strictly read-only and immutable.
Q2	B	React enforces unidirectional (parent-to-child) flow.
Q3	B	Returns an array containing the snapshot value and dispatcher.

Q#	Answer	Explanation / Core Concept
Q4	B	Makes function parameters readable and explicit.
Q5	B	Setters trigger React to schedule a re-render.
Q6	B	Callback avoids stale closure issues by reading pending state.
Q7	B	Standard camelCase naming convention.
Q8	B	React state acts as the

Q#	Answer	Explanation / Core Concept
		single source of truth for input values.
Q9	B	children contains whatever is nested inside tags.
Q10	B	Array spread [...arr, item] creates a fresh array instance.
Q11	B	Re-renders re-instantiate local variables and bypass UI updates.

Q#	Answer	Explanation / Core Concept
Q12	B	Consolidates multiple setter triggers into a single render.
Q13	C	Spreading previous state retains untouched properties.
Q14	A	Adding parentheses calls the function instantly during render.
Q15	B	Standard method to prevent full page reloads on form submit.

Q#	Answer	Explanation / Core Concept
Q16	B	Normalizes native event attributes across different web browsers.
Q17	A	Standard ES6 default parameter assignment syntax.
Q18	B	State is lifted up to the common ancestor component.
Q19	B	Handlers closing over past state snapshots.

Q#	Answer	Explanation / Core Concept
Q20	B	Props configure from outside; state manages local changes.